Tracking the Lifecycle of Kubernetes Configuration Smells Across Versions: Preliminary Evidence (Milestone 3)

Group 4
ENGR 5570G Software Maintenance and Evolution, Ontario Tech University
Rohan Muslekar (101006689), Ankit Suman (101041764), Yasaminzahra Valitabar (101001242),
Bayan Sayyadifar (101035809), Seyedeh Rojan Razavi (101001141)

***Abstract*: We follow the configuration smells that kube-linter and kube-score report in the Kubernetes manifests of 7 open-source projects across every minor release since late 2022 (123 versions). Smells are sticky: 82.3% of the 821 instances present at a project's first sampled release are still present at its last, and none was removed and later reintroduced. Of 147 removals, 72.1% happened because the object itself was deleted. Commit tracing of every fix and introduction shows that fixes arrive in a few batch hardening commits (32 fixes, 12 commits), mostly on security smells and a median of 1.8 years after the smell appeared, and that most new smells on existing objects are side effects of fixing something else. Rendering every kustomization in the study, our strongest validity threat, changes which smells are visible but none of these findings, and sampling every release instead of five changes almost nothing. A blind LLM rater and an AI-assisted team rating agree with the category map on 59 and 58 of 62 checks ($\kappa = 0.93, 0.91$). Remaining work is the two-rater human category agreement and making rendering the pipeline default.**

I. Research Questions and Progress

Questions. RQ1 (prevalence, supporting): which smell categories occur in released Kubernetes manifests, and how often? RQ2 (lifecycle, primary): once a smell instance appears, does it persist, disappear, or reappear in later releases, and when it disappears, is the smell fixed or the object removed? RQ3 (explanation, primary): which changes cause smells to appear and disappear? The scope is the manifests a project ships in its repository, analyzed at released tags. Table I links each question to the evidence that answers it.

TABLE I
Research questions and the evidence that answers them

RQ	Evidence in this report	Status
RQ1	Instances per category and tool (Table III), per system (Table II), most frequent checks (Table IV)	answered
RQ2	Transitions (Table III), survival per system (Fig. 1) and per category (Table V), removal causes, smell age, reappearance over all minor releases	answered
RQ3	77 transitions traced to commits (Table VI), including every fix and every introduction on an existing object, with cases	answered

Changes since Milestone 2. Each change answers a weakness that Milestone 2 reported.

1) **Every minor release instead of five.** Five tags over four years can hide a smell that is removed and re-added between two sampled tags, so zero reappearances was only a lower bound. We now sample every minor release, 123 versions.
2) **Rendering check.** Milestone 2 linted kustomize bases and patches as stored, and 84 of its removals were flux2 patch fragments, so the deletion finding rested on unrendered input. We rendered every kustomization in the study with `kubectl kustomize` at every tag and compared (Section IV).
3) **Category validation.** The check-to-category map was written by one author; the procedure for two human raters is in place; a blind automated rater and an AI-assisted rater 1 have rated all 62 checks.
4) **Rename detection.** A renamed object used to count as one removal and one introduction; tracking now flags probable renames.
5) **Commit tracing** grows from 4 spot-checked fixes to every fix and every introduction on an existing object (RQ3).
6) **RQ4 (image CVEs) dropped**, as feedback put the weight on the lifecycle.

II. Methods Executed

Systems and versions. Table II lists the 7 systems. A candidate is included when it ships plain Kubernetes YAML in-tree, had commits in the last six months and has at least 500 stars, and has at least five release tags over at least 12 months; 4 failed and are recorded with the reason in `data/repos.json`. We take the latest patch of every minor release first published since 2022-10-22, ignore tags dated after 2026-10-22 so the sample does not move when a project publishes a patch, and keep every Milestone 2 tag (`python3 -m kst.select_tags work/<repo> 2022-10-22 --all --until 2026-10-22 --keep <M2 tags>`). `data/versions.csv` records the commit of each version; all 123 are valid.

Pipeline. For each tag, `scripts/collect.sh` checks out the tag, keeps the plain Kubernetes YAML under the

manifest path and logs every skipped file with its reason (388 kustomization files over all tags, 382 of them in flux2). It records the objects defined at the tag and runs kube-linter v0.8.3 [1] (all built-in checks, pinned configuration) and kube-score v1.20.0 [2]. A tag whose analyzer output is empty or not JSON is invalid and left out, so a crash does not look like removed smells. The 62 observed checks map to four categories seeded from Kubernetes misconfiguration and configuration-smell work [3], [4], [5], or to *other* (style and opinionated defaults, excluded). An instance is keyed by (project, tool, kind, namespace, name, check); each pair of consecutive tags labels it persisted, introduced, removed or reappeared, and the object inventory separates removals where the object was deleted from fixes.

Executed since Milestone 2.

- **Dense tags** (PR #10): the tag selection above; `kst.subset` cuts any run back to the five Milestone 2 tags for comparison.
- **Rendering check** (PR #15): a new script, `scripts/render_check.py`, renders every kustomization that no other kustomization lists (15 in flux2 at the first tag, one in microservices-demo, the only projects that use kustomize), drops the plain files they use, and runs the same inventory, analyzers and tracking at all 22 of their tags. Ankit's change (PR #14) keeps rendered paths machine-independent.
- **Category validation** (PR #11): `kst.category_validation prepare` writes blank sheets of all 62 checks with the definitions but not the map; `calculate` gives agreement and Cohen's kappa. An LLM (Claude Opus 5.5) rated one sheet blind (PR #19), from the definitions and the analyzers' documentation only, never the repository or the map. Rater 1's sheet (PR #21) says it is AI-assisted.
- **Rename detection** (PR #12): an object that disappears and one that appears between consecutive tags are flagged as a probable rename when they share the kind and the same non-empty set of (tool, check); histories are not merged.
- **Commit tracing** (PR #13, PR #17): for each fix and each introduction on an existing object, `git log <from>..<to> -- <file>` finds the commit; we read its diff and record the commit, the configuration change, an evidence link and a cause in `data/rq3_investigations.csv`.
- **Reproduction**: CI reruns 110 tests and the offline analysis on every change and fails if `out/` differs. A fresh clone from GitHub at the data merge (PR #16) recollected all 123 tags on Fedora in 17 min: `out/` and `data/versions.csv` were byte-identical and all tests passed; raw files differed only in kube-linter's timestamp and kube-score's check order.

Deviations from plan. Rendering inside the pipeline (`kst/render.py`, `RENDER=1`, now merged in PR #20) was not ready for the integration run, so the main results use stored manifests and the rendering check, which covers all 388 skipped kustomization files, gives the rendered results beside them. Rater 1's sheet came back AI-assisted and rater 2's has not; human agreement moves to Milestone 4. Helm, checkov and Trivy are not run: no included system needs Helm, and Trivy served only RQ4.

Rerunning. From https://github.com/Rohan-Muslekar/kube-smell-tracker: `./tools/install.sh && make all` rebuilds every result, `python3 -m kst` reruns the analysis offline from the committed `raw/`, and `python3 scripts/render_check.py <repo> --clone <clone> --out <dir> --field tags` on the PR #15 branch reruns the rendering check (`results/render_check.md`). The Milestone 2 dataset is kept at git tag `milestone-2-data`. The environment (Fedora 44, Go 1.26.9, Python 3.14.8 standard library, kubectl 1.36.2 with kustomize 5.8.1) is listed in the README. No LLM is used except in the category ratings.

III. Preliminary Findings

RQ1: prevalence. Over the 123 versions the analyzers report 862 smell instances (483 kube-linter, 379 kube-score; 276 object–category pairs when both tools are merged). Resource and reliability smells are the most common (420, 48.7%), mostly missing probes, limits and replica settings, followed by security (261, 30.3%), image (76) and hygiene (105). Per system the count ranges from 56 (Tekton pipeline) to 239 (istio's bookinfo samples, Table II), roughly with the number of workloads shipped. The smells concentrate on a few checks (Table IV): eight account for 44.0% of all instances, and six of them occur in at least six of the seven systems. The same omissions recur whatever the project does: no ephemeral-storage request or limit, fewer than three replicas, no fixed user and group ID, no network policy. The exception, `use-namespace` (3 systems), fires on manifests that leave the namespace to the installer.

RQ2: lifecycle. 676 of 821 first-tag instances (82.3%) are present at the last tag, and 676 of all 862 instances (78.4%) are present at every sampled tag. Across 123 versions, 0 instances reappeared. Of 147 removals, 106 (72.1%) are object deletions and 41 keep the object; 9 of those are a kube-score artifact (it stops comparing probes once they use gRPC), leaving 32 real fixes. The variation is large (Fig. 1): argo-cd and istio keep 98–100% of their first-tag smells and four more systems about 90%, while Knative serving keeps 65% and flux2 12%, losing most at once when it deletes a directory. Without flux2 only 34.9% of removals are deletions, so the deletion share is a property of two systems, not of all.

Survival also differs by category (Table V). Security smells survive least (73.8%) because 26 of the 32 real fixes remove security smells; resource smells get 6, and no image or hygiene smell was ever fixed, only deleted with its object. Hygiene smells survive most (94.1%). A chi-square test of independence between category and survival over the 821 first-tag instances is significant ($\chi^2(3) = 24.6$, $p < 0.001$) but the effect is small (Cramér's $V = 0.17$). It holds without flux2

TABLE II

STUDY SYSTEMS AND PER-SYSTEM RESULTS (SMELLS EXCLUDING *other*). SURV.: SHARE OF FIRST-TAG INSTANCES PRESENT AT THE LAST TAG. PARENTHESES: REMOVALS WITH THE OBJECT DELETED; INTRODUCTIONS WITH A NEW OBJECT.

System	Manifest path	Tags	First–last tag (years)	Inst.	Surv.	Removed	Introd.
microservices-demo	`kubernetes-manifests`	6	v0.5.2–v0.10.7 (2023–2026)	182	89.0%	20 (0)	1 (0)
kube-prometheus	`manifests`	8	v0.12.0–v0.19.0 (2023–2026)	80	91.2%	7 (0)	0 (0)
argo-cd	`manifests/install.yaml`	16	v2.5.22–v3.5.6 (2023–2026)	127	98.2%	2 (0)	13 (1)
flux2	`manifests`	16	v0.36.0–v2.9.6 (2022–2026)	97	11.6%	84 (84)	2 (2)
istio	`samples/bookinfo/platform/kube`	16	1.16.7–1.31.2 (2023–2026)	239	100.0%	0 (0)	10 (0)
serving	`config/core`	15	v0.36.4–v0.50.0 (2023–2026)	81	65.4%	28 (22)	0 (0)
pipeline	`config`	46	v0.41.3–v1.17.0 (2023–2026)	56	90.2%	6 (0)	15 (13)

TABLE III

SMELL INSTANCES AND TRANSITIONS PER CATEGORY AND TOOL (KL = KUBE-LINTER, KS = KUBE-SCORE). PARENTHESES: INTRODUCTIONS WITH A NEW OBJECT; REMOVALS WITH THE OBJECT DELETED.

Category	Tool	Inst.	Base	Persist	Introd.	Removed	Reapp.
Security & privilege	KL	147	126	1996	21 (5)	39 (22)	0
Security & privilege	KS	114	111	1427	3 (2)	25 (16)	0
Resource & reliability	KL	216	212	3009	4 (4)	28 (24)	0
Resource & reliability	KS	204	199	2483	5 (3)	33 (22)	0
Image & supply chain	KL	25	24	334	1 (1)	6 (6)	0
Image & supply chain	KS	51	48	636	3 (1)	10 (10)	0
Hygiene	KL	95	91	1007	4 (0)	4 (4)	0
Hygiene	KS	10	10	112	0 (0)	2 (2)	0
Total		862	821	11004	41 (16)	147 (106)	0

TABLE IV

THE EIGHT MOST FREQUENT CHECKS. SYS.: SYSTEMS WHERE THE CHECK OCCURS; SURV.: SHARE OF ITS FIRST-TAG INSTANCES PRESENT AT THE LAST TAG.

Check	Tool	Inst.	Sys.	Surv.
`use-namespace`	KL	63	3	100.0%
`container-ephemeral-storage-request-and-limit`	KS	51	7	88.0%
`minimum-three-replicas`	KL	48	7	91.5%
`container-security-context-user-group-id`	KS	48	7	77.1%
`no-rolling-update-strategy`	KL	47	7	91.3%
`pod-networkpolicy`	KS	45	6	86.0%
`deployment-replicas`	KS	39	6	97.3%
`non-isolated-pod`	KL	38	5	83.8%

TABLE V

LIFECYCLE PER CATEGORY, BOTH TOOLS. FIRST: INSTANCES PRESENT AT THE FIRST TAG. REAL FIXES EXCLUDE THE ANALYZER ARTIFACT. PARENTHESES AS IN TABLE III.

Category	First	Surv.	Removed	Real fixes	Introd.
Security & privilege	237	73.8%	64 (38)	26	24 (7)
Resource & reliability	411	85.2%	61 (46)	6	9 (7)
Image & supply chain	72	77.8%	16 (16)	0	4 (2)
Hygiene	101	94.1%	6 (6)	0	4 (0)

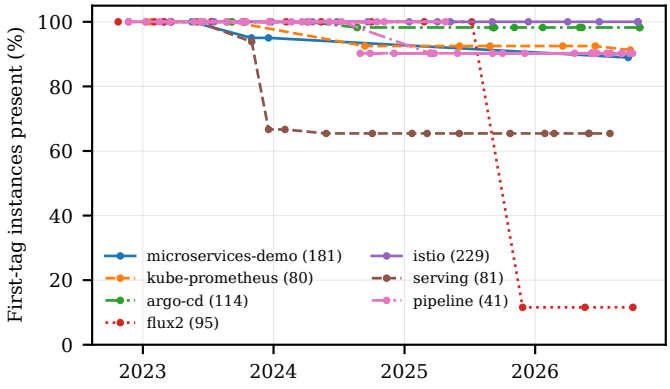


Fig. 1. Share of first-tag smell instances still present at each later release, per system (first-tag instances in parentheses), plotted at the release's commit date.

($\chi^2(3) = 17.3$, $p < 0.001$) but not with the 268 object–category pairs as the unit ($\chi^2(3) = 6.7$, $p = 0.08$): several smells on one object tend to go together, so instances are not independent. We treat the category difference as suggestive. Fixes are also slow: a real fix comes a median of 1.8 years after the smell first appears (0.4–3.7 years; 8 of 32 within a year), and the 715 instances present at the last tag have been present for a median of 3.2 years, most since the first sampled release.

RQ3: what causes transitions. We traced 77 transitions to commits (`data/rq3_investigations.csv`): all 41 fixes, all 25 introductions on objects that already existed, and 4 with a new object or Service, plus 4 deletions and 5 persisting smells from the spot-check. 32 fixes are deliberate configuration changes and 9 the analyzer artifact. They come from only 12 commits (Table VI), and four of them give 24; one microservices-demo commit (20585fa) gives every service its own ServiceAccount and removes 11 `default-service-account` smells at once. Fixes arrive in batch hardening changes, not one smell at a time. kube-prometheus is the exception: its fixes come from 7 commits, each hardening one component's security context.

New smells on existing objects are mostly side effects of fixing something else: 23 of 25, and all 25 come from 7 commits. In argo-cd the fix for advisory GHSA-9766-5277-j5hr enabled Redis authentication: a password read from a secret in four workloads, a Role that reads secrets and an init container together add ten smells. In istio, `runAsUser` was removed from every bookinfo Deployment because OpenShift rejects fixed UIDs; the images now set the user, so the containers still run as non-root, but a manifest-only analyzer reports ten new `run-as-non-root` smells. In microservices-demo, pinning `busybox:latest` to a digest changed the default pull policy

TABLE VI
COMMITS BEHIND THE 32 REAL FIXES, PER SYSTEM

System	Commits	Fixes	What changed
microservices-demo	1	11	a ServiceAccount per service
kube-prometheus	7	7	`runAsGroup`, `runAsNonRoot`, no RBAC wildcards; one component per commit
pipeline	1	6	read-only root filesystem
serving	2	6	probes on the domain-mapping controllers; host anti-affinity
argo-cd	1	2	read-only root filesystem on Redis

and introduced a kube-score smell: fixing one supply-chain smell created another.

Analyzer overlap. The tools agree on 134 of 241 object–category pairs (55.6%), 78.5% without hygiene, echoing the disagreement reported between static analyzers [6]; we therefore report per-tool counts. They differ most on hygiene, where 73 of 83 pairs are kube-linter only (`use-namespace` and label checks kube-score does not have), and on image smells, where 21 of 46 are kube-score only (its pull-policy check).

IV. INTERPRETATION AND VALIDITY

What the evidence supports. Within these 7 projects, a smell present in a release is very likely still present years later, and when one disappears it is more often because the object went away than because someone fixed it. Fixes that do happen are deliberate and batched. We do not claim this for Helm-only projects or for live cluster state, and the deletion share holds for flux2 and Knative serving, not for each system.

Comparison with prior work. The persistence matches what Rahman et al. found for security smells in Puppet scripts, where a hard-coded secret lived a median of 20 months [5]; our real fixes come a median of 1.8 years after the smell appears. Dockerfile studies report smell counts that slowly decline over time [7], but an aggregate count cannot tell a fix from a deleted file. Rosa et al. [8] therefore inspected the commits that removed Dockerfile smells and found that developers mostly act on changes that improve image size and build time. In our manifests most removals are deletions and the deliberate fixes are security hardening instead. Work on Helm charts followed outdated images and security risks across chart versions [9], not individual smell instances, which is what our tracking adds.

Unexpected results. Zero reappearance, now over every minor release, suggests that a smell once fixed stays fixed, as fixes are deliberate hardening commits. The kube-score probe artifact shows that an analyzer can make a smell disappear without any configuration change; we keep these out of the fix count. We also did not expect whole categories to go unfixed: no image or hygiene smell was fixed in four years. Both concern choices a project leaves to whoever installs it (the image pull policy, the target namespace), so maintainers have little reason to change them in the shipped manifests, while security settings are part of what the project itself promises.

Table VII lists each threat and the check already run.

TABLE VII
VALIDITY THREATS AND THE CHECKS ALREADY RUN

Threat	Check run	Result
Unrendered kustomize	every kustomization rendered at every tag (PR #15)	all findings hold; 11 hygiene smells were false positives
Sparse sampling	every minor release, 123 tags (PR #16)	no reappearance; 2 removals change cause
Analyzer change read as a fix	every fix traced to a commit (PR #13, PR #17)	9 removals with no configuration change excluded
Rename read as fix	probable-rename detection (PR #12)	0 smell instances flagged
Crash read as removal	output validation per tag	all 123 tags valid
Dependent instances	chi-square on object–category pairs	category effect $p = 0.08$, read as suggestive
Non-reproducible results	fresh clone from GitHub	`out/` byte-identical
Single-author category map	LLM (PR #19); AI-assisted rater 1 (PR #21)	59, 58/62 ($\kappa = 0.93$, 0.91); human pending

TABLE VIII
RENDERING CHECK AT EVERY TAG OF THE TWO PROJECTS WITH KUSTOMIZATIONS (S: STORED, R: RENDERED). ALL: THE WHOLE STUDY WITH BOTH PROJECTS RENDERED.

	flux2		microserv.-demo		All	
	S	R	S	R	S	R
Smell instances	97	162	182	166	862	911
Surv. to last tag	11.6%	24.2%	89.0%	88.6%	82.3%	79.6%
Removed	84	116	20	19	147	178
Deleted share	100.0%	100.0%	0.0%	0.0%	72.1%	77.5%
Real fixes	0	0	11	10	32	31
Reappeared	0	0	0	0	0	0

Strongest threat: unrendered manifests. Linting kustomize bases and patches as stored can both invent smells on fragments and miss objects that only exist after rendering. We ran the check on every kustomization in the study (Table VIII). Rendering raises flux2's smells from 97 to 162: the six controller Deployments are remote resources, so stored manifests never contained them, and their smells mostly persist at every tag. The 11 hygiene smells, all on network policies and selectors before kustomize adds the common labels, disappear, so they were false positives. The deletion finding does not change: all 116 rendered removals are deletions, when flux2 removed `manifests/integrations` at v2.9.6. In microservices-demo, rendering only drops the `loadgenerator` Deployment, which the kustomization comments out (16 instances). With both projects rendered, survival is 79.6%, the deletion share 77.5%, 31 real fixes remain and nothing reappears: every finding holds.

Check run: sparse sampling. Cutting the 123-version run back to the five Milestone 2 tags reproduces Milestone 2 byte for byte. The 88 extra tags add no instance and no reappearance and leave survival at 82.3%. Only two removals change cause: Knative serving's `domain-mapping` Deployment gained liveness and readiness probes at v0.37.6 and was deleted later, which five tags saw only as a deletion.

TABLE IX
PLAN TO THE FINAL REPORT (PRESENTATIONS NOV 27 AND DEC 4, REPORT DEC 11)

Task	Owner	Due	Fallback
Rendering as the pipeline default (`kst/render.py`)	Rojan, Ankit	Nov 20	keep the rendering check (Table VIII) as the rendered results
Category ratings, kappa, decision ledger	Ankit, Rojan (raters); Yasaminzahra	Nov 18	one rating only: report it against the map as a consistency check
Case studies for the presentation	Bayan	Nov 24	the three cases in Section III
Presentation	all	Nov 27	present the results in this report; rendered and agreement results as work in progress
Recheck every number against `out/`; repeat the fresh clone	Rohan	Dec 4	every number is generated by `make_tables.py`, so a change in `out/` is rerun, not retyped
Final report and artifact	all	Dec 11	items not done by Dec 4 are reported as limitations, with the fallback evidence above

TABLE X
MEMBER CONTRIBUTIONS SINCE MILESTONE 2 (PRS AND FILES IN HTTPS://GITHUB.COM/ROHAN-MUSLEKAR/KUBE-SMELL-TRACKER)

Member	Completed work and linked artifacts
Rohan Muslekar	Dense tag selection and `kst.subset` (PR #10); rendering check (PR #15); integration run (PR #16); remaining RQ3 tracing and survival figure (PR #17); LLM category rating (PR #19); task plan (PR #9); PR reviews; this report
Ankit Suman	Rendered-input support in `analyze.sh` and `normalize.py` with tests (PR #14); AI-assisted rater 1 sheet (PR #21)
Yasaminzahra Valitabar	Category-validation sheets, agreement and decision ledger (PR #11); probable-rename detection with tests and `kst.rename_report` (PR #12)
Bayan Sayyadifar	Commit tracing of all 39 Milestone 2 fixes and 3 introductions, with evidence links (PR #13); case studies
Seyedeh Rojan Razavi	Kustomize renderer `kst/render.py`, `RENDER=1` and tests (PR #20); rater 2

Alternative explanation. Persistence need not mean neglect. Some smells are deliberate defaults: istio's bookinfo samples are a demo kept minimal on purpose, and a maintainer may accept a missing CPU limit. Commit tracing agrees: smells go when a project decides to harden a component, so survival measures how rarely that happens, not how many smells are mistakes. We therefore read RQ2 as describing what happens to reported smells, not as a defect rate.

Category construct validity. The category map is one author's. The blind LLM rater and rater 1's AI-assisted sheet agree with it on 59 and 58 of 62 checks (Cohen's $\kappa = 0.93, 0.91$) and with each other on 61, so rater 1 is not independent of an automated rating. All differences are map-*other* checks (`no-node-affinity`, `restart-policy`, `schema-validation`; rater 1 adds `dnsconfig-options`). Their categories give 962 and 1013 instances, 82.6% and 82.9% survival, 74.1% and 75.0% deleted and no reappearance; no finding hinges on them.

Other threats. Rename detection flags 8 instances, no real smell among them (0), so renames do not inflate removals. With pinned analyzers, changes come from the configuration (bar the probe artifact). Seven projects and two tools limit generality; per-tool counts avoid double-counting.

Implications. For researchers, a manifest lifecycle study should separate deletions from fixes and render kustomize first: without the object inventory the 106 deletions would count as fixes, over three times the 32 real ones, and without rendering flux2's six controller Deployments are never analyzed. For tool builders, istio's user set in the image (invisible to a manifest check) and the kube-score probe artifact both produce lifecycle events with no configuration change.

V. REMAINING WORK AND CONTRIBUTIONS

Scope reductions. RQ4 (image CVEs) is dropped. Helm-only projects stay excluded, so the results cover projects that ship plain or kustomize YAML. checkov is not added.

Toward Milestone 4. Table IX gives each remaining task an owner, a date and a fallback. The rendering check and the two ratings show that neither analysis item (rendering by default, human category agreement) can change RQ2 or RQ3, so the plan holds if both fall back.

REFERENCES

[1] "KubeLinter, v0.8.3," https://github.com/stackrox/kube-linter, StackRox.
[2] "kube-score, v1.20.0," https://github.com/zegl/kube-score.
[3] A. Rahman, S. I. Shamim, D. B. Bose, and R. Pandita, "Security misconfigurations in open source Kubernetes manifests: An empirical study," *ACM Transactions on Software Engineering and Methodology*, vol. 32, no. 4, pp. 1–36, 2023.
[4] T. Sharma, M. Fragkoulis, and D. Spinellis, "Does your configuration code smell?" in *Proceedings of the 13th International Conference on Mining Software Repositories (MSR)*, 2016, pp. 189–200.
[5] A. Rahman, C. Parnin, and L. Williams, "The seven sins: Security smells in infrastructure as code scripts," in *2019 IEEE/ACM 41st International Conference on Software Engineering (ICSE)*, 2019, pp. 164–175.
[6] V. Lenarduzzi, F. Pecorelli, N. Saarimäki, S. Lujan, and F. Palomba, "A critical comparison on six static analysis tools: Detection, agreement, and precision," *Journal of Systems and Software*, vol. 198, p. 111575, 2023.
[7] K. Eng and A. Hindle, "Revisiting Dockerfiles in open source software over time," in *2021 IEEE/ACM 18th International Conference on Mining Software Repositories (MSR)*, 2021, pp. 449–459.
[8] G. Rosa, F. Zappone, S. Scalabrino, and R. Oliveto, "Fixing Dockerfile smells: An empirical study," *Empirical Software Engineering*, vol. 29, p. 108, 2024.
[9] A. Zerouali, R. Opdebeeck, and C. D. Roover, "Helm charts for Kubernetes applications: Evolution, outdatedness and security risks," in *2023 IEEE/ACM 20th International Conference on Mining Software Repositories (MSR)*, 2023, pp. 523–533.